CSE 242 Assignment 1 — Part 3

PCA derivation, visualizations, and analysis

This report accompanies **Assignment1.py**. The separate handwritten PDF contains Part 1 and Part 2 derivations. The experiment below uses 500 reproducible observations (NumPy random seed 0).

Q1. Variance of a projection and covariance matrix

Let X be a d-dimensional random column vector, let its mean be mu = E[X], and let u be a fixed direction with $u^T u = 1$. The projected random variable $Y = u^T X$ is a scalar. By linearity of expectation:

```
E[Y] = E[u^T X] = u^T E[X] = u^T mu.
```

The covariance matrix is defined as $Sigma = E[(X - mu)(X - mu)^T]$. Its (i,j)-th element is $Cov(X_i, X_j)$, the expected product of the two centered variables. Expanding the product gives:

```
Sigma = E[XX^T] - mu mu^T.
```

Starting with the scalar definition of variance, substitute E[Y] and regroup the scalar square:

```
Var(u^T X) = E[(u^T X - u^T mu)^2]

= E[(u^T (X - mu))^2]

= E[u^T (X - mu)(X - mu)^T u]

= u^T E[(X - mu)(X - mu)^T] u

= u^T Sigma u.
```

The identity itself holds for any fixed u; unit length makes variances comparable across directions without changing their scale.

Q2. Two correlated Gaussian features

Generate X using **numpy.random.default_rng(0).multivariate_normal**, with population mean (2, -1) and population covariance matrix:

```
Sigma_pop = [[3.0, 1.2], [1.2, 1.0]].
```

The population correlation is 1.2 / sqrt(3 x 1) = **0.6928**. The observed sample correlation is **0.6992**. Its positive sign agrees with the upward-sloping, elongated point cloud: larger first-feature values tend to be associated with larger second-feature values. Correlation is not perfect, so the cloud has a nonzero width.

Q3. PCA implementation from scratch

Step 1 — Center. Calculate the sample mean and subtract it from every observation. This shifts the data cloud to the origin without changing its covariance.

```
mu_hat = (1/n) sum_i x_i ; z_i = x_i - mu_hat.
```

Step 2 — Compute covariance. Form the unbiased sample covariance matrix S; dividing by n - 1 matches NumPy sample covariance convention.

```
S = (1/(n - 1)) sum_i z_i z_i^T = Z^T Z / (n - 1).
```

Step 3 — Find eigenpairs. The sample projected variance along unit u is $u^T Su$, directly corresponding to Q1. To maximize the variance, impose $u^T u = 1$ with a Lagrange multiplier:

```
L(u, lambda) = u^T S u - lambda (u^T u - 1).

grad_u L = 2 S u - 2 lambda u = 0 => S u = lambda u.
```

For a normalized eigenvector, $u^T Su$ = lambda. Hence the largest eigenvalue identifies the maximum-variance direction, PC1. For a symmetric covariance matrix, the eigenvectors can be chosen orthonormal; PC2 captures the remaining variance.

Step 4 — Sort and project. Use **np.linalg.eigh** for the symmetric matrix, sort eigenpairs by decreasing eigenvalue, and compute the new coordinates as Z times the matrix of principal component vectors. This is a NumPy implementation, not a fitted PCA library.

```
scores = Z @ components.
```

The eigenvector sign is arbitrary. Rotating into the PC basis makes the component scores uncorrelated; their sample variances equal the corresponding eigenvalues.

Computed sample statistics

Statistic	Value
Sample mean	(2.152399, -0.934645)
Sample covariance	[[2.916156, 1.165663], [1.165663, 0.953025]]
Eigenvalues (PC1, PC2)	(3.458480, 0.410701)
Sample correlation	0.699222

Q4. Explained variance and dimensionality reduction

The total variance is the trace of the sample covariance, equivalently the sum of its eigenvalues. Each explained-variance ratio equals that component eigenvalue divided by the sum:

```
EVR_j = lambda_j / sum_k lambda_k.

PC1: 3.458480 / 3.869181 = 89.39%.

PC2: 0.410701 / 3.869181 = 10.61%.
```

Keeping just PC1 reduces each observation from two coordinates to one and preserves **89.39%** of sample variance while discarding **10.61%**. Reconstructing from PC1 gives an average squared Euclidean error of **0.409880** per observation. Keeping both components retains 100% of the variance, but does not reduce dimensionality.

This illustrates the compression-versus-information trade-off: fewer coordinates reduce storage and downstream computational costs, while retaining more components preserves more sample variance. Low-variance directions

can still carry information useful for a later prediction task, so the chosen dimension should depend on the application rather than the ratio alone.

Visualizations and interpretation

The left panel shows the original correlated data and orthogonal principal directions through the sample mean. PC1 follows the long diagonal axis of the cloud; PC2 follows its much narrower perpendicular axis. The middle panel shows the centered data after projection into uncorrelated PC coordinates. The right panel shows individual and cumulative explained variance.

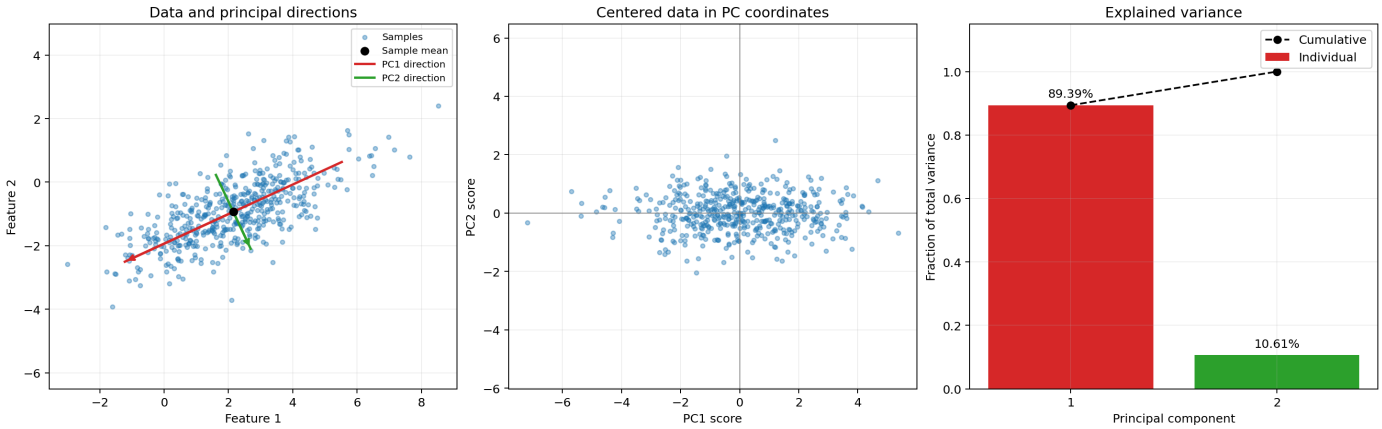


Figure: Original samples and PC directions; transformed scores; explained variance.